0424. Longest Repeating Character Replacement

ory: Sliding Window | LeetCode: https://leetcode.com/problems/longest-repeatin

Problem

- S has uppercase letters. You may change at most k characters. Return the longest substring that can become all one letter.
- Example: "AABABBA", k = 1 → 4.

Intuition

- A window is fixable if (window length - count of its most frequent letter) <= k.
- Grow the window to the right; if it becomes unfixable, slide the left edge by one.
- MaxFreq never needs to decrease: a smaller maxFreq could never produce a longer answer than one already recorded.

Brute Force

- Check every substring with counts: O(26 n^2).

Optimal Approach

- Count[26], l = 0, maxFreq = 0.
- For r: count[s[r]]++, maxFreq = max(maxFreq, count[s[r]]).
- If (r - l + 1) - maxFreq > k: count[s[l]]--, l++.
- Best = max(best, r - l + 1).

Worked Example

```
s = "AABABBA", k = 1.

r=0 A win=A       maxF=1 ok  best=1
r=1 A win=AA      maxF=2 ok  best=2
r=2 B win=AAB     maxF=2 ok  best=3
r=3 A win=AABA    maxF=3 ok  best=4
r=4 B win=AABAB   5-3=2>1 -> drop A  win=ABAB
r=5 B win=ABABB   5-3=2>1 -> drop A  win=BABB
r=6 A win=BABBA   5-3=2>1 -> drop B  win=ABBA
best = 4
```

Java Solution

```java
public class LongestRepeatingCharacterReplacement {
    public int characterReplacement(String s, int k) {
        int[] count = new int[26];
        int l = 0, maxFreq = 0, best = 0;
        for (int r = 0; r < s.length(); r++) {
            maxFreq = Math.max(maxFreq, ++count[s.charAt(r) - 'A']);
            if (r - l + 1 - maxFreq > k) count[s.charAt(l++) - 'A']--;
            best = Math.max(best, r - l + 1);
        }
        return best;
    }
}
```

Complexity

- Time: O(n). Space: O(26).

Pitfalls

- Recomputing the true max on every shrink is correct but O(26n); the stale maxFreq trick is still correct.
- Use if, not while, for the shrink: the window only ever slides, it never needs to shrink by more than one.